



Test and Reliability

Chapter 4: Boolean Satisfiability

Prof. Dr. Ralf Wimmer
Dr. Tobias Seufert

Summer Term 2026



References



References

- A. Biere, M. J. H. Heule, H. van Maaren, T. Walsh: [Handbook of Satisfiability](#), 2nd edition, IOS Press, 2021.
- D. Knuth: The Art Of Computer Programming, Volume 4, Fascile 6 (Actually: Chapter 7.2.2.2 with 320 pages), Addison-Wesley 2016
- Tons of workshop, conference, and journal papers ...
- Different solver implementations like Minisat, Cadical, Kissat, and many more.
- Jan Burchard: Applying Advanced SAT-Based Techniques to Circuit Testing. PhD Thesis, University of Freiburg, 2018.

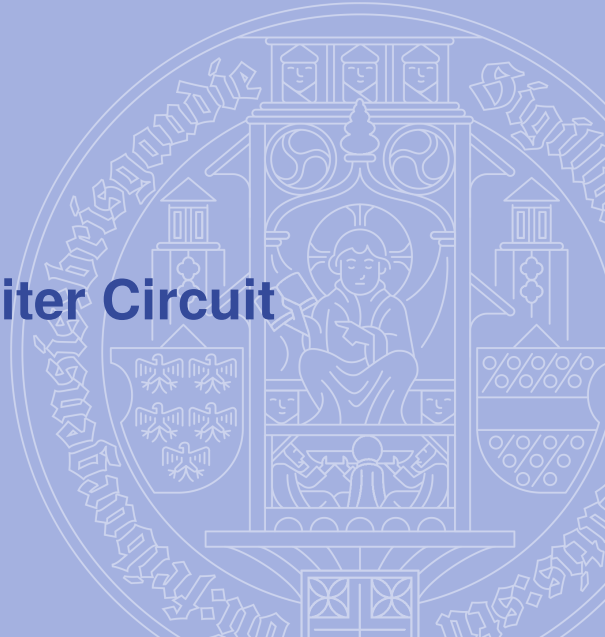
Satisfiability Problem of Propositional Logic

Definition (SAT-Problem)

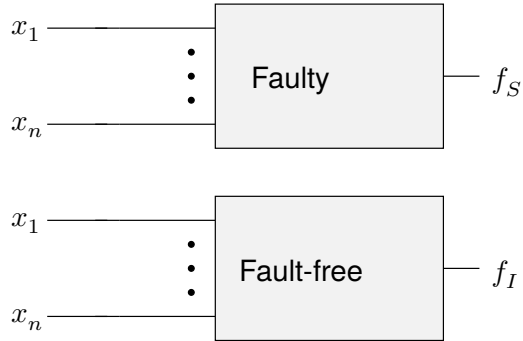
Let F be a propositional logic formula. The question to answer is: is F satisfiable? That is, exists an assignment $\mathcal{A}$ for the variables in F so that $\mathcal{A}(F) = 1$ hold?

- Techniques for solving instances of the SAT-problem are called SAT-algorithms or also SAT-solvers

Construction of the Miter Circuit

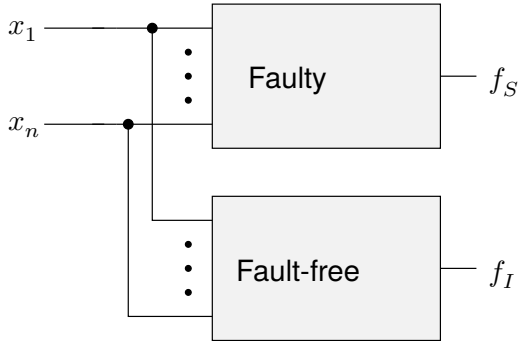


Construction of the Miter Circuit



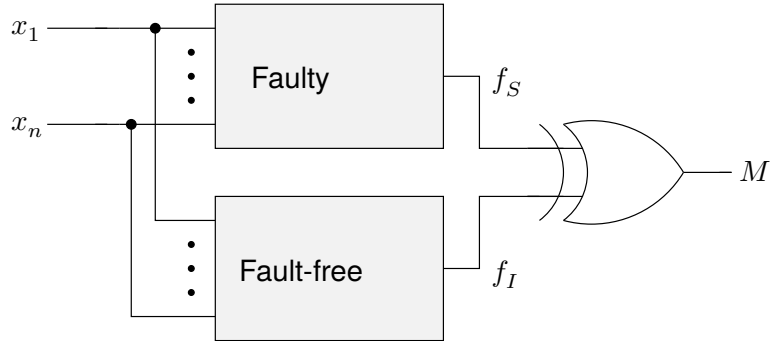
⇒ Connect corresponding inputs

Construction of the Miter Circuit



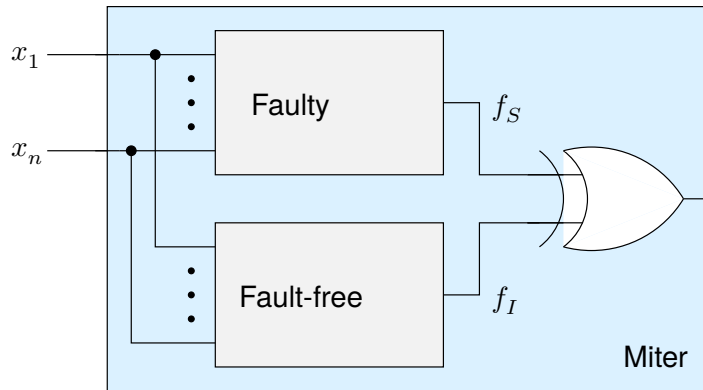
⇒ Link corresponding outputs by EXOR gates

Construction of the Miter Circuit



⇒ Miter circuit

Construction of the Miter Circuit



literal : x_i or $\neg x_i$
clause : $(x_1 \vee \neg x_5 \vee x_7)$

CNF : conjunction of clauses

$$C_1 \wedge C_2 \wedge C_3 \wedge \dots \wedge$$

M

$$\neg \neg x = x$$

$$\neg(a \wedge b) = \neg a \vee \neg b$$

$$\neg(a \vee b) = \neg a \wedge \neg b$$

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$$

$\Rightarrow M = 1$ for some input assignment $\Leftrightarrow$ Fault-free circuit & faulty circuit are **not** equivalent

Tseitin Transformation



Tseitin Transformation

$$\begin{array}{c}
 a \text{ ---} \\
 b \text{ ---}
 \end{array}
 \text{---} \bigwedge \text{---} c$$

$$\begin{aligned}
 & c \equiv (a \wedge b) \\
 & c \rightarrow (a \wedge b) \wedge (a \wedge b) \rightarrow c \\
 & \text{CNF} \rightarrow ((a \wedge b) \vee \neg c) \wedge (c \vee \neg (a \wedge b)) \\
 & \quad (a \vee \neg c) \wedge (b \vee \neg c) \wedge (c \vee \neg a \vee \neg b)
 \end{aligned}$$

Direct transformation of a formula into conjunctive normal form often yields an exponential blow-up.

Alternative technique: Tseitin transformation of a formula F :

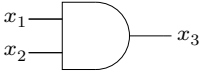
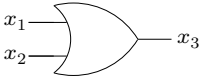
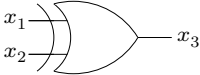
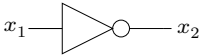
- Define for F an **equisatisfiable** CNF F' that is satisfiable iff F is satisfiable.
- For each gate output insert an additional variable; in general the CNF F' will have variables which do not occur in F
- For each gate realize a “characteristic function” in CNF, which evaluates to 1 for every possible consistent signal configuration
- Put together the individual gates using an AND connection to obtain the final CNF formula
- Fix the output variable of the circuit to be 1

⇒ **Tseitin transformation**

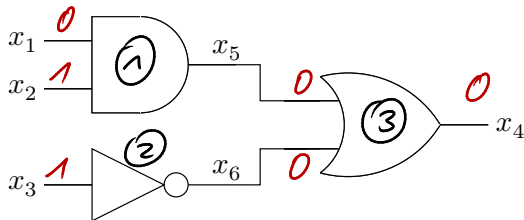
$$\begin{array}{c|c|c}
 a & b & c \\
 \hline
 1 & 1 & 0
 \end{array}$$

$$\begin{aligned}
 & a \wedge b \\
 & (a \vee \neg c)(b \vee \neg c)(\neg a \vee \neg b \vee c) \\
 & \quad \wedge c
 \end{aligned}$$

Tseitin Transformation

Gates	Function	CNF formula
	$x_3 \equiv x_1 \wedge x_2$	$(\neg x_3 \vee x_1) \wedge (\neg x_3 \vee x_2) \wedge (x_3 \vee \neg x_1 \vee \neg x_2)$
	$x_3 \equiv x_1 \vee x_2$	$(x_3 \vee \neg x_1) \wedge (x_3 \vee \neg x_2) \wedge (\neg x_3 \vee x_1 \vee x_2)$
	$x_3 \equiv x_1 \oplus x_2$	$(\neg x_3 \vee x_1 \vee x_2) \wedge (\neg x_3 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee \neg x_1 \vee x_2) \wedge (x_3 \vee x_1 \vee \neg x_2)$
	$x_2 \equiv \neg x_1$	$(x_2 \vee x_1) \wedge (\neg x_2 \vee \neg x_1)$

Tseitin Transformation

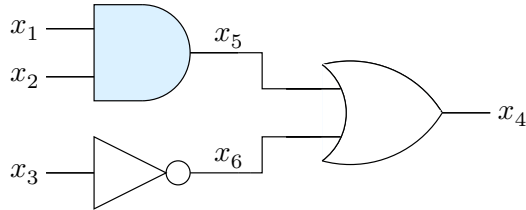


→ satisfies first 3 lines of the formula

$$F_{SK} = \underline{(x_1 \wedge x_2) \vee \neg x_3}$$

$$F_{SK}^{CNF} = \begin{array}{l} \underline{(\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (x_5 \vee \neg x_1 \vee \neg x_2)} \quad \textcircled{1} \\ \underline{(x_6 \vee x_3) \wedge (\neg x_6 \vee \neg x_3)} \quad \textcircled{2} \\ \underline{(x_4 \vee \neg x_5) \wedge (x_4 \vee \neg x_6) \wedge (\neg x_4 \vee x_5 \vee x_6)} \quad \textcircled{3} \\ \underline{(x_4)} \end{array}$$

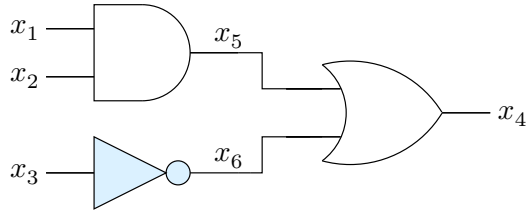
Tseitin Transformation



$$F_{SK} = (x_1 \wedge x_2) \vee \neg x_3$$

$$\begin{aligned} F_{SK}^{CNF} = & (\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (x_5 \vee \neg x_1 \vee \neg x_2) \wedge \\ & (x_6 \vee x_3) \wedge (\neg x_6 \vee \neg x_3) \wedge \\ & (x_4 \vee \neg x_5) \wedge (x_4 \vee \neg x_6) \wedge (\neg x_4 \vee x_5 \vee x_6) \wedge \\ & (x_4) \end{aligned}$$

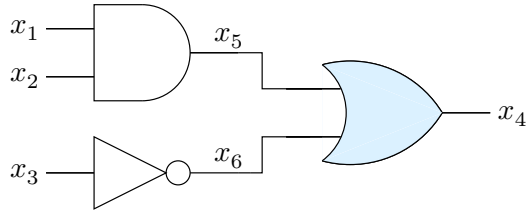
Tseitin Transformation



$$F_{SK} = (x_1 \wedge x_2) \vee \neg x_3$$

$$\begin{aligned} F_{SK}^{CNF} = & (\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (x_5 \vee \neg x_1 \vee \neg x_2) \wedge \\ & (x_6 \vee x_3) \wedge (\neg x_6 \vee \neg x_3) \wedge \\ & (x_4 \vee \neg x_5) \wedge (x_4 \vee \neg x_6) \wedge (\neg x_4 \vee x_5 \vee x_6) \wedge \\ & (x_4) \end{aligned}$$

Tseitin Transformation



$$F_{SK} = (x_1 \wedge x_2) \vee \neg x_3$$

$$F_{SK}^{CNF} = (\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (x_5 \vee \neg x_1 \vee \neg x_2) \wedge \\ (x_6 \vee x_3) \wedge (\neg x_6 \vee \neg x_3) \wedge \\ (x_4 \vee \neg x_5) \wedge (x_4 \vee \neg x_6) \wedge (\neg x_4 \vee x_5 \vee x_6) \wedge \\ (x_4)$$

Tseitin Transformation

As long as for the CNF representation of each single gate only a constant number of clauses is required, the number of clauses in the final CNF will be linear in the number of gates in the circuit (the same holds for the size of the formula)

Resolution



Resolution

Definition (Resolution)

Let C_1 and C_2 be two clauses and L be a literal with the following property: $L \in C_1$ and $\neg L \in C_2$. Then one can compute the clause R

$$R = (C_1 - \{L\}) \cup (C_2 - \{\neg L\})$$

that is denoted as the **resolvent** of the clauses C_1 and C_2 over L . The notation $R = C_1 \otimes_L C_2$ is commonly used.

Resolution

Example:

- $C_1 = (x_1, \boxed{x_2}, x_3), C_2 = (x_4, \boxed{\neg x_2})$
 $C_1 \otimes_{x_2} C_2 = (x_1, x_3, x_4)$

- $C_3 = (\boxed{x_4}, \boxed{x_2}, x_3), C_4 = (x_4, \boxed{\neg x_2})$
 $C_3 \otimes_{x_2} C_4 = (\boxed{x_3}, x_4)$

- $C_5 = (x_4, \boxed{x_2}), C_6 = (\neg x_4, \boxed{\neg x_2})$
 $C_5 \otimes_{x_2} C_6 = (x_4, \neg x_4)$

Tautology → can always be ignored

- $(x_4, \neg x_4)$ is satisfied for every assignment of x_4 , and is called a tautology clause.

Resolution

$$R = C_1 \oplus_x C_2$$
$$x \in C_1, \neg x \in C_2$$

Lemma (Resolution Lemma)

Let F be a CNF formula and R be the resolvent of two clauses C_1 and C_2 from F . Then F and $F \cup \{R\}$ are equivalent: $F \equiv F \cup \{R\}$.

Assume F satisfiable with satisfying assignment v .

$\rightarrow v$ satisfies C_1 and v satisfies C_2

$$\textcircled{1} \quad v(x) = 1$$

Resolution

Definition

Let F be a CNF formula. Then $Res(F)$ is defined as

$$Res(F) = F \cup \{R \mid R \text{ is the resolvent of two clauses in } F\}.$$

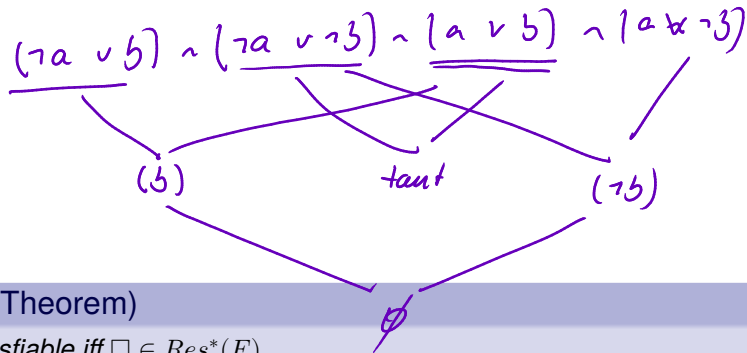
Moreover we define:

$$Res^0(F) = F$$

$$Res^{t+1}(F) = Res(Res^t(F)) \text{ for } t \geq 0$$

$$Res^*(F) = \lim_{t \geq 0} Res^t(F)$$

Resolution



Theorem (Resolution Theorem)

A CNF formula F is unsatisfiable iff $\square \in \text{Res}^*(F)$.

Modern SAT Algorithms



Modern SAT Algorithms

Basic techniques of today's SAT solvers:

- Preprocessing
- Alternation between ...
 - Choose the next decision variable, decide its value
 - Boolean Constraint Propagation / Unit Propagation
 - If applicable, conflict analysis & backtracking
- At some fixed points of the search process
 - Unlearning (some conflict clauses)
 - Restarts
- In case of satisfiable CNF formula
 - Output the model found, that is the satisfying valuation

$$\neg a \sim (\underline{a} \vee \neg b) \sim (\underline{a} \vee \underline{b} \vee \neg c)$$

$\hookrightarrow a := 0 \rightarrow b := 0 \rightarrow c := 0$

BCP

implications

Modern SAT Algorithms

Main routine of sequential SAT algorithms

```
bool SEQUENTIALSATENGINE(CNF F)
{
    if (PREPROCESSCNF(F) == CONFLICT)                // Preprocessing the CNF formula
        { return UNSATISFIABLE; }                    // Problem unsatisfiable

    while (true)
    {
        if (DECIDENEXTBRANCH())                        // Choice of the next unassigned variable
        {
            while (BCP() == CONFLICT)                  // Boolean Constraint Propagation
            {
                BLevel = ANALYZECONFLICT();              // Conflict analysis
                if (BLevel > 0) { BACKTRACK(BLevel); }    // Cancel the "incorrect" assignment
                else return UNSATISFIABLE;               // Problem unsatisfiable
            }
        }
        else return SATISFIABLE;                        // All variables assigned, problem satisfiable
    }
}
```

Preprocessing



Modern SAT Algorithms

Main routine of sequential SAT algorithms

```
bool SEQUENTIALSATENGINE(CNF F)
{
    if (PREPROCESSCNF(F) == CONFLICT)           // Preprocessing the CNF formula
    { return UNSATISFIABLE; }                   // Problem unsatisfiable

    while (true)
    {
        if (DECIDENEXTBRANCH())                  // Choice of the next unassigned variable
        {
            while (BCP() == CONFLICT)             // Boolean Constraint Propagation
            {
                BLevel = ANALYZECONFLICT();        // Conflict analysis
                if (BLevel > 0) { BACKTRACK(BLevel); } // Cancel the "incorrect" assignment
                else return UNSATISFIABLE;         // Problem unsatisfiable
            }
        }
        else return SATISFIABLE;                  // All variables assigned, problem satisfiable
    }
}
```

Preprocessing

- Goal
 - To make the given CNF formula (much) easier in preparation for the actual search process
- Observation from practical experience
 - As a rule of thumb, the size of a formula is related to the time necessary for the SAT algorithm to solve it
 - Reduction by over 75% (number of clauses and variables)
- It is very important to find a good compromise between the additional effort required by preprocessing and the expected saving during the search process

Preprocessing Techniques

- Unit propagation
- Unit Propagation Lookahead
- Self-subsuming resolution
- Elimination by clause distribution
- Variable elimination by substitution
- Forward subsumption
- Backward subsumption

Preprocessing

Unit Propagation (if omitted, then immediately performed during solving process):

- Identify unit clauses in the CNF
- Assign units, simplify
- ... until the CNF is free from unit clauses

Preprocessing

Unit Propagation Lookahead (UPLA):

- Fix a variable x_i to 0, check implications; then change its value to $x_i = 1$, check implications. Simplify the formula exploiting the following consequences:

- $(x_i = 0 \rightarrow \text{conflict}) \wedge (x_i = 1 \rightarrow \text{conflict}) \Rightarrow \text{UNSAT}$

- $(x_i = 0 \rightarrow \text{conflict}) \Rightarrow x_i = 1$

- $(x_i = 1 \rightarrow \text{conflict}) \Rightarrow x_i = 0$

- $(x_i = 0 \rightarrow x_j = 1) \wedge (x_i = 1 \rightarrow x_j = 1) \Rightarrow x_j = 1$

- $(x_i = 0 \rightarrow x_j = 0) \wedge (x_i = 1 \rightarrow x_j = 0) \Rightarrow x_j = 0$

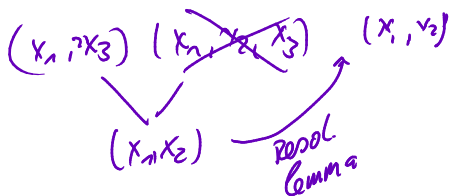
- $(x_i = 0 \rightarrow x_j = 0) \wedge (x_i = 1 \rightarrow x_j = 1) \Rightarrow x_i \equiv x_j$

- $(x_i = 0 \rightarrow x_j = 1) \wedge (x_i = 1 \rightarrow x_j = 0) \Rightarrow x_i \equiv \neg x_j$

Unit Propagation Lookahead (UPLA):

- Advantage
 - Built on top of the components already available in the solver
- Disadvantages
 - Requires binary clauses in the original formula
 - Necessary to extend the model when e. g. $x_i \equiv x_j$ is detected and all the occurrences of x_i are substituted with x_j .
 - In general quite time consuming, in particular if all the variables are tested.

Preprocessing



Self-Subsuming Resolution:

- Original formula
 - $F = (x_1 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3) \wedge \dots$
- Resolution applied to the first two clauses
 - $(x_1 \vee \neg x_3) \otimes_{x_3} (x_1 \vee x_2 \vee x_3) = (x_1 \vee x_2)$
 - $\Rightarrow (x_1 \vee x_2)$ **subsumes** $(x_1 \vee x_2 \vee x_3)$
 - $\Rightarrow$ Replace $(x_1 \vee x_2 \vee x_3)$ with $(x_1 \vee x_2)$
- Simplified formula
 - $F' = (x_1 \vee \neg x_3) \wedge (x_1 \vee x_2) \wedge \dots$
- Saving
 - 1 Literal

Preprocessing

$$F = P \cup N \cup W$$

$\uparrow$ all clauses with x $\nwarrow$ all clauses with $\neg x$ $\swarrow$ Best

Variable Elimination by Resolution:

- Original formula P
 - $F = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_1 \vee \neg x_2)$
- Variable elimination applied to x_1 leads to
 - $F' = (x_2 \vee x_3) \wedge (\neg x_3 \vee \neg x_2)$
- Saving
 - 1 Variable, 2 Clauses, 4 Literals
- It is applied only if it leads to a simplification of the formula (reduction of size, "black magic numbers" ...).

$$(P \otimes_x N) \cup W$$

equisatisf. to the original formula F

x_1	x_2	x_3
0	1	0

$$F \Rightarrow \exists x : F = F_{|x=0} \vee F_{|x=1}$$

$$((P \setminus \{x\}) \cup W) \vee ((N \setminus \{\neg x\}) \cup W)$$

Preprocessing

Variable Elimination by Substitution:

$$x_5 \equiv (x_1 \wedge x_2)$$

- Original formula

$$F = (\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (x_5 \vee \neg x_1 \vee \neg x_2) \wedge (x_4 \vee \neg x_5) \wedge (\neg x_4 \vee x_5 \vee x_6)$$

- The first three clauses represent an AND gate (Tseitin transformation)

$$[(\neg x_5 \vee x_1) \wedge (\neg x_5 \vee x_2) \wedge (x_5 \vee \neg x_1 \vee \neg x_2)] \leftrightarrow [x_5 \equiv x_1 \wedge x_2]$$

- Removing the first three clauses, and replacing the occurrences of x_5 by $x_1 \wedge x_2$ in the other clauses leads to

$$F' = (x_4 \vee \neg(x_1 \wedge x_2)) \wedge (\neg x_4 \vee (x_1 \wedge x_2) \vee x_6) \leftarrow$$

- Transformation into CNF

$$F'' = (x_4 \vee \neg x_1 \vee \neg x_2) \wedge (\neg x_4 \vee x_1 \vee x_6) \wedge (\neg x_4 \vee x_2 \vee x_6)$$

- Saving: 1 variable, 2 clauses, 3 literals

- It is applied only if it leads to a simplification of the formula** (reduction of size, “black magic numbers” ...).

- Procedure for OR, NAND, etc. gates similar

Preprocessing

Forward Subsumption:

- Test if a clause generated during one of the above preprocessing techniques is subsumed by one clause of the initial formula

Backward Subsumption:

- Test if a clause generated during one of the above preprocessing techniques subsumes one (or more) clauses of the initial formula

⇒ Remove all the clauses subsumed

$$x_1 \rightarrow c_1, \dots$$
$$\neg x_2 \rightarrow c_5, \dots$$

$$(l_1, l_2, l_3)$$

$$C \text{ subsumes } C' \text{ iff}$$
$$C \subseteq C'$$

Preprocessing

$$C = (\underline{x_1}, x_2, \neg x_3, x_4) \begin{cases} (\neg x_1, \neg x_2, \neg x_3) & D_1 \\ (\neg x_1, \neg x_4) & D_2 \\ (\neg x_1, x_3, x_4) & D_3 \end{cases}$$

Blocked Clause Elimination:

- Let C be a clause and $\ell \in C$ a literal.
- C is blocked by ℓ iff for all clauses D with $\neg \ell \in D$ holds:
 $C \otimes_{\ell} D$ is a tautology.
- Blocked clauses can be deleted without changing satisfiability.

⚠ Be careful with the satisfying assignment returned for $F \setminus \{C\}$!

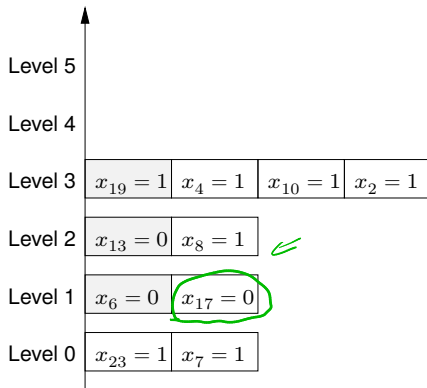
Preprocessing

Many more techniques available that are not discussed here, but, hopefully, the main ideas should have become clear ...

Decision Stack



Decision Stack

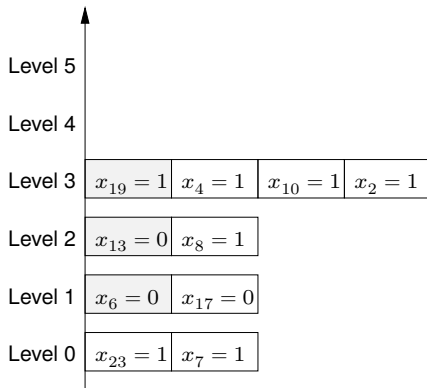


(x_{13}, x_8)
 $(\underline{x_{13}}, \underline{x_{17}}, \underline{x_8})$

- Central data structure of modern SAT algorithms
- Decision Stack stores the order of the executed assignments
- If a model for a CNF formula could be found, the decision stack stores the satisfying assignment

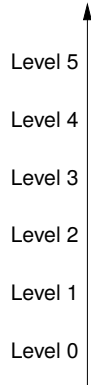
$(\underline{x_8}, \underline{x_{17}})$
 $\underline{0}$

Decision Stack



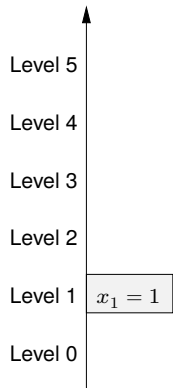
- Each variable assignment has an associated decision level
- Decision level is initialized with 0; before a decision is made, it is incremented by one; backtracking decrements the decision level appropriately.
- Decision Level 0 plays a special role: it stores only implications from unit clauses in the original formula, but no decisions.
- A conflict on decision level 0 means that the CNF is unsatisfiable.

Decision Stack



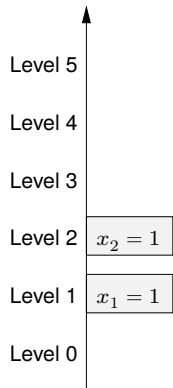
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



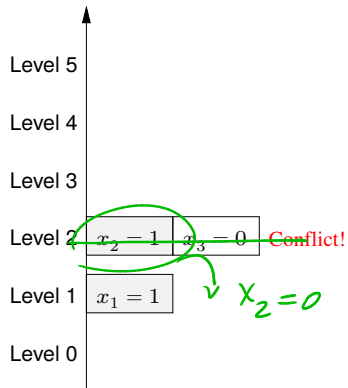
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



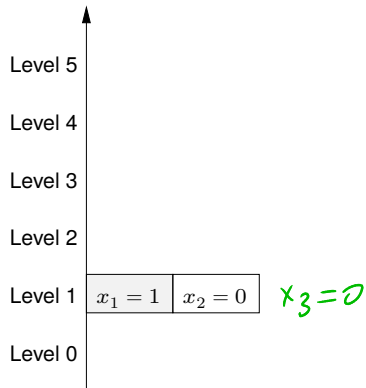
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

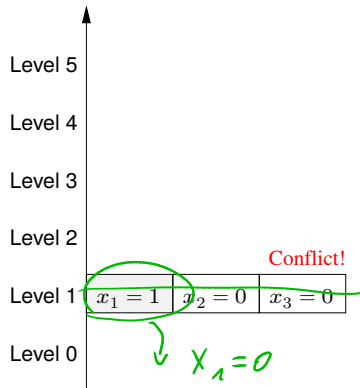
Decision Stack



$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

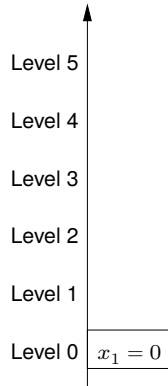
(Note: In the original image, the terms $(\neg x_1, x_2, \neg x_3)$ and $(\neg x_1, x_2, x_3)$ are underlined in green, and a green lightning bolt symbol is drawn above the x_2 in the second term.)

Decision Stack



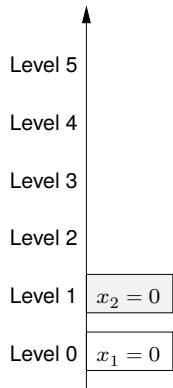
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



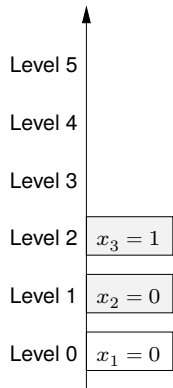
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



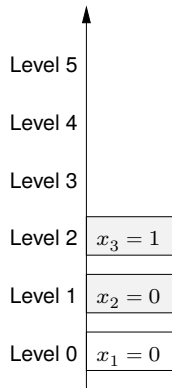
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



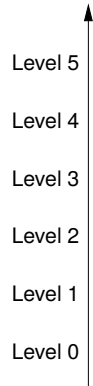
$$(\neg x_1, \neg x_2, \neg x_3) \wedge (\neg x_1, \neg x_2, x_3) \wedge (\neg x_1, x_2, \neg x_3) \wedge (\neg x_1, x_2, x_3) \wedge (x_1, \neg x_2, \neg x_3)$$

Decision Stack



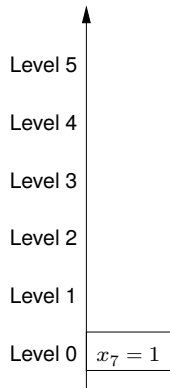
⇒ Formula satisfiable with, e.g., $x_1 = 0, x_2 = 0, x_3 = 1$

Decision Stack



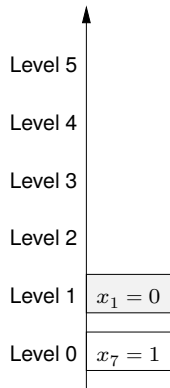
$$(x_1, x_2) \wedge (x_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \neg x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



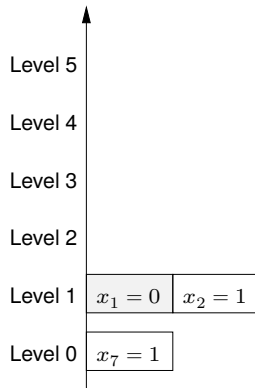
$$(x_1, x_2) \wedge (x_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \neg x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



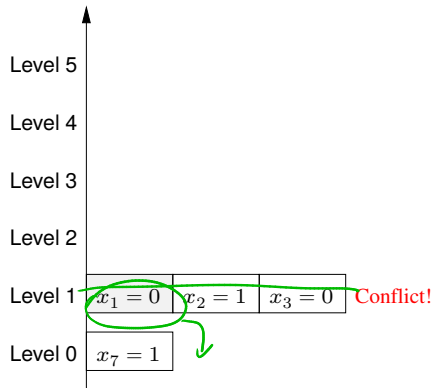
$$\underline{(x_1, x_2)} \wedge \underline{(x_1, \neg x_3)} \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \neg x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



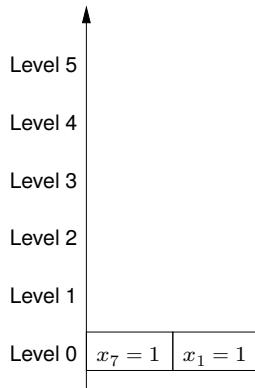
$$(x_1, x_2) \wedge (\textcolor{red}{x}_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \textcolor{red}{\neg} x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



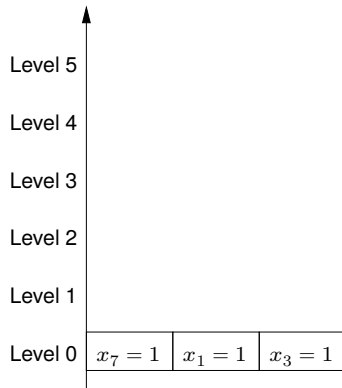
$$(x_1, x_2) \wedge (x_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (\textcolor{red}{x}_3, \textcolor{red}{\neg} x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



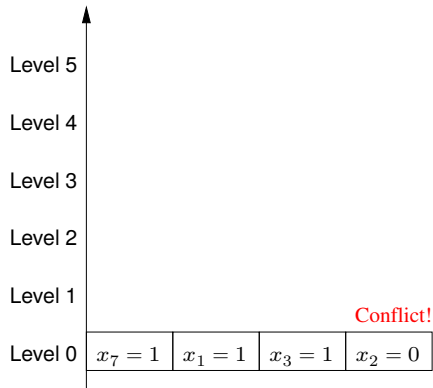
$$(x_1, x_2) \wedge (x_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \neg x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



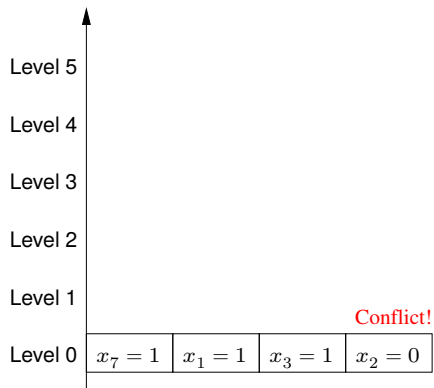
$$(x_1, x_2) \wedge (x_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \neg x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



$$(x_1, x_2) \wedge (x_1, \neg x_3) \wedge (\neg x_1, x_3) \wedge (\neg x_1, \neg x_2) \wedge (x_3, \neg x_2) \wedge (\neg x_3, x_2) \wedge (x_7)$$

Decision Stack



⇒ Formula unsatisfiable due to conflict on decision level 0

Decision Heuristics



Modern SAT Algorithms

Main routine of sequential SAT algorithms

```
bool SEQUENTIALSATENGINE(CNF F)
{
    if (PREPROCESSCNF(F) == CONFLICT)
        { return UNSATISFIABLE; }
    while (true)
    {
        if (DECIDENEXTBRANCH())
        {
            while (BCP() == CONFLICT)
            {
                BLevel = ANALYZECONFLICT();
                if (BLevel > 0) { BACKTRACK(BLevel); }
                else return UNSATISFIABLE;
            }
        }
        else return SATISFIABLE;
    }
}
```

decision heuristic

// Preprocessing the CNF formula
// Problem unsatisfiable

// Choice of the next unassigned variable

// Boolean Constraint Propagation

// Conflict analysis
// Cancel the "incorrect" assignment
// Problem unsatisfiable

// All variables assigned, problem satisfiable

Decision Heuristics

- Has the role of choosing the next **Decision Variable**
- Comparable with “case distinction” in the DLL algorithm
- Affects significantly the search process!
- Modern SAT algorithms do not test whether the CNF formula is already satisfied during the search, rather it is indirectly guaranteed from assigning all the variables without running into a conflict
 - Example: $F = (x_1, x_2, x_3) \wedge (\neg x_1, x_4)$
 - ⇒ A satisfying valuation is for example $x_1 = 1, x_4 = 1$
 - ⇒ Today's procedures do not test if $x_1 = x_4 = 1$ already satisfies all the clauses, but assign the other variables without generating a conflict (e. g., $x_2 = x_3 = 0$) before they output *SATISFIABLE*

Decision Heuristics

Variable State Independent Decaying Sum (VSIDS) (zChaff)

- Today's standard method used by every SAT solver
- Computation complexity determined by #Variables
- Each literal ℓ has an activity counter N_ℓ
- Initially, N_ℓ equals the number of occurrences of ℓ in the original CNF.
- For each literal ℓ in a new learned clause C , the activity is incremented by one.
- The unassigned literal ℓ with the highest activity N_ℓ is chosen as decision literal and set to true.
- No update is needed for assignments and backtracks, since values do not depend on variable state.

Variable State Independent Decaying Sum (VSIDS) (zChaff)

- Periodically, the activities are “normalized”, i. e., divided by a constant factor
 - ⇒ After the normalization, the recently learned clauses have a higher weight in comparison with the clauses learned before that last normalization
 - ⇒ Takes into account the “history” of the search process
- Several optimizations possible (black magic ...)
 - By which amount should the activity be incremented?
 - How often should the normalization take place?
 - By which factor should the activity scores be divided?

Boolean Constraint Propagation



Modern SAT Algorithms

Main routine of sequential SAT algorithms

```
bool SEQUENTIALSATENGINE(CNF F)
{
    if (PREPROCESSCNF(F) == CONFLICT)                // Preprocessing the CNF formula
        { return UNSATISFIABLE; }                    // Problem unsatisfiable

    while (true)
    {
        if (DECIDENEXTBRANCH())                        // Choice of the next unassigned variable
        {
            while (BCP() == CONFLICT)                  // Boolean Constraint Propagation
            {
                BLevel = ANALYZECONFLICT();              // Conflict analysis
                if (BLevel > 0) { BACKTRACK(BLevel); }    // Cancel the "incorrect" assignment
                else return UNSATISFIABLE;                // Problem unsatisfiable
            }
        }
        else return SATISFIABLE;                        // All variables assigned, problem satisfiable
    }
}
```

Boolean constr. prop.

Boolean Constraint Propagation

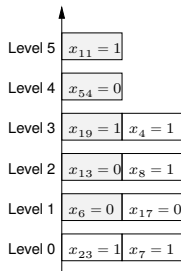
- Tasks
 - Detecting all the implications from the assignment of a variable
 - Detecting conflicts
- Comparable to the repeated application of the unit rule in the DLL algorithm
- Efficient implementation mandatory, because roughly 80% of the runtime of a SAT algorithm is spent in the BCP routine

Boolean Constraint Propagation

General procedure:

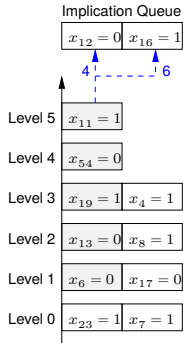
- After every variable assignment, identify the implications that have arisen, and push them into the implication queue
- As long as there are items in the implication queue ...
 1. Remove the first element from the queue
 2. Assign to each implied variable its forced truth value
 3. Check which consecutive implications arise, and push them into the implication queue

Boolean Constraint Propagation



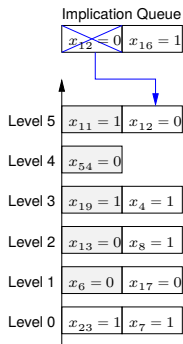
$$\begin{aligned}
 F = & \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \\
 & \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots
 \end{aligned}$$

Boolean Constraint Propagation



$$\begin{aligned}
 F = & \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots \\
 & \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots
 \end{aligned}$$

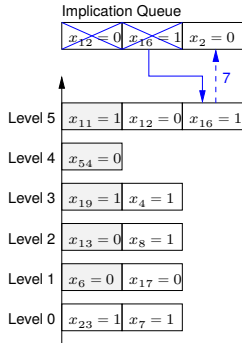
Boolean Constraint Propagation



$$F = \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots$$

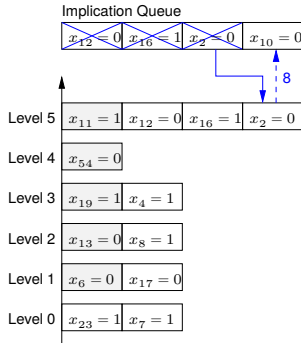
$$\underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots$$

Boolean Constraint Propagation



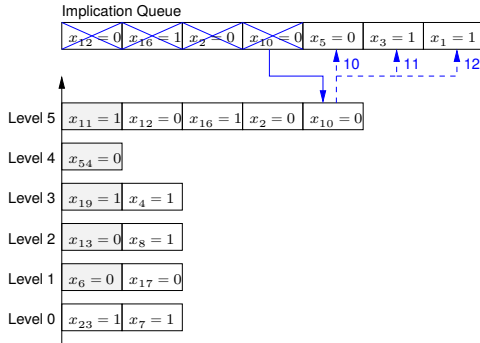
$$\begin{aligned}
 F = & \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots \\
 & \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots
 \end{aligned}$$

Boolean Constraint Propagation



$$\begin{aligned}
 F = & \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots \\
 & \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots
 \end{aligned}$$

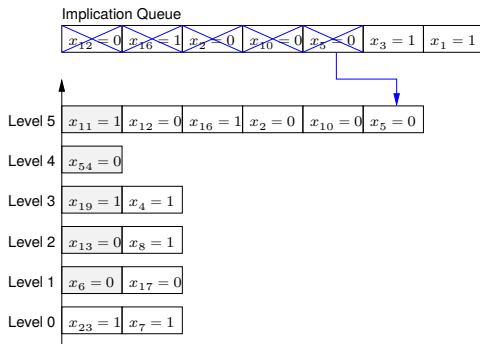
Boolean Constraint Propagation



$$F = \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots$$

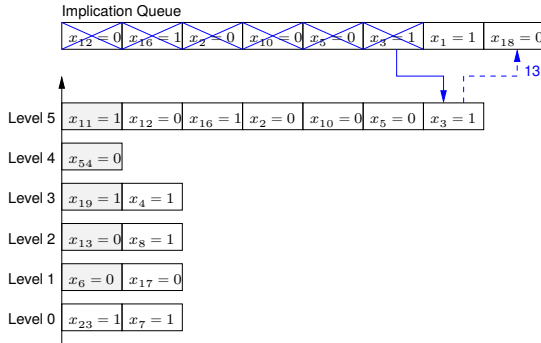
$$\underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots$$

Boolean Constraint Propagation



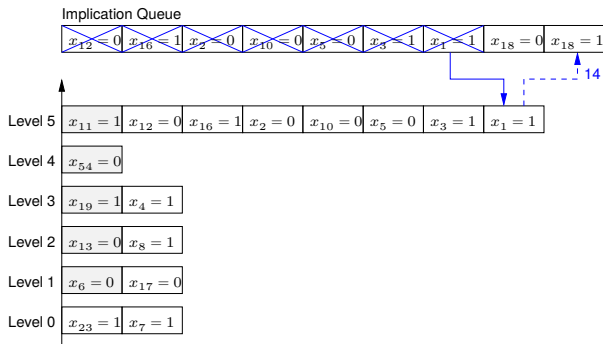
$$\begin{aligned}
 F = & \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots \\
 & \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots
 \end{aligned}$$

Boolean Constraint Propagation



$$\begin{aligned}
 F = & \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots \\
 & \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots
 \end{aligned}$$

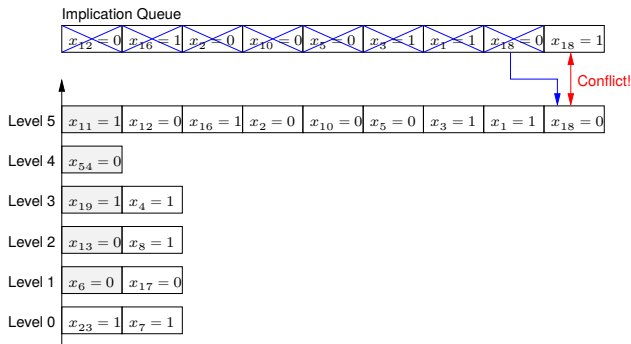
Boolean Constraint Propagation



$$F = \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots$$

$$\underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots$$

Boolean Constraint Propagation



$$F = \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \dots$$

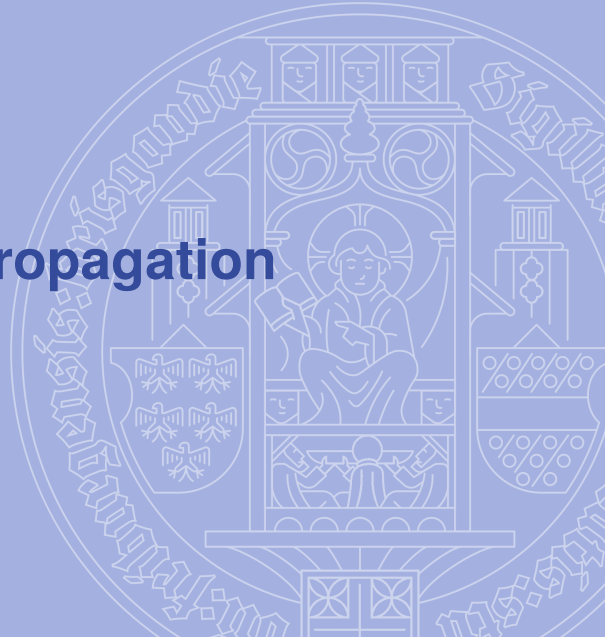
$$\underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots$$

Boolean Constraint Propagation

- Tasks
 - Detecting all the implications from the assignment of a variable
 - Detecting conflicts
- Possible approaches for the implementation of a BCP routine
 - Watched Literals / 2-Literal Watching Scheme

Boolean Constraint Propagation

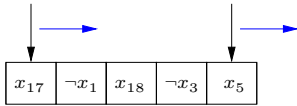
Watched Literals



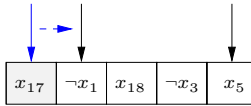
Watched Literals

- **Idea:** As long as a clause contains two unassigned (or one satisfied) literal, it can not be a unit clause and the exact number of unassigned literals is not important
- For each clause mark two different literals
- Invariant
 - The watched literals of a clause are either unassigned or satisfy the clause
- Advantages in comparison to counter-based schemes
 - Update operations only when necessary, i. e., when an assignment “breaks” the invariant
 - One list for each variable and polarity (like above) containing only the clauses currently watching that literal
- Disadvantages
 - The literals of a clause are checked several times

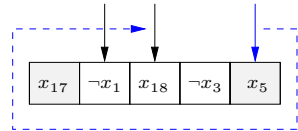
Watched Literals



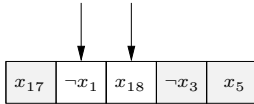
(a) Initial state



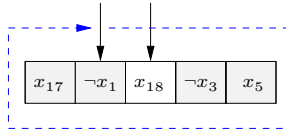
(b) $x_{17} = 0$



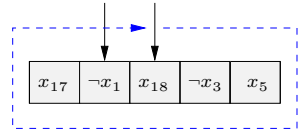
(c) $x_5 = 0$



(d) $x_3 = 1$



(e) $x_1 = 1 \Rightarrow x_{18} = 1$



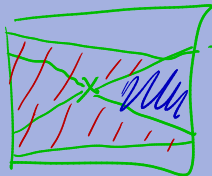
(f) $x_{18} = 0 \Rightarrow \text{Conflict!}$

Watched Literals

- Possible optimizations
 - Store the watched literals in the first two positions (memory “cells”) of a clause
 - Makes it possible to access to the “second” watched literal of a clause fast, as well as its status
 - If the second watched literal satisfies the clause, it is not necessary to find a replacement for the first one, if this is in turn falsified
- Watched Literal schemes have been used in all the modern SAT solvers in the last years.

Conflict Analysis and Backtracking

cone-of-influence reduction



outputs
on which
fault might
be visible

Modern SAT Algorithms

Main routine of sequential SAT algorithms

```
bool SEQUENTIALSATENGINE(CNF F)
{
    if (PREPROCESSCNF(F) == CONFLICT)                // Preprocessing the CNF formula
        { return UNSATISFIABLE; }                    // Problem unsatisfiable

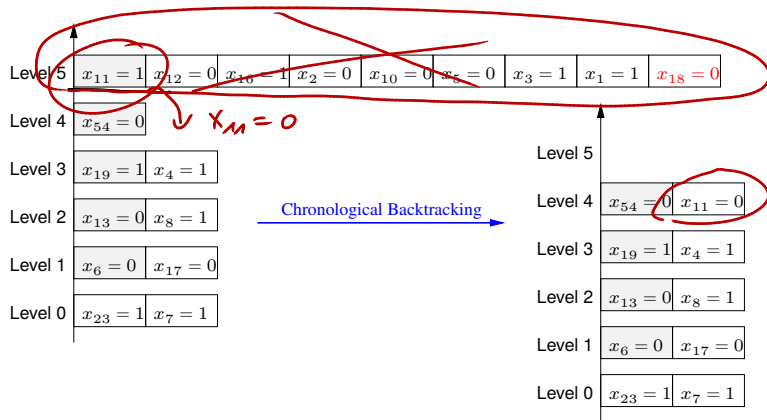
    while (true)
    {
        if (DECIDENEXTBRANCH())                        // Choice of the next unassigned variable
        {
            while (BCP() == CONFLICT)                  // Boolean Constraint Propagation
            {
                BLevel = ANALYZECONFLICT();              // Conflict analysis
                if (BLevel > 0) { BACKTRACK(BLevel); }    // Cancel the "incorrect" assignment
                else return UNSATISFIABLE;               // Problem unsatisfiable
            }
        }
        else return SATISFIABLE;                        // All variables assigned, problem satisfiable
    }
}
```

Conflict Analysis and Backtracking

DLL algorithm:

- The combination of the previously made branches (decisions) is always seen as the origin of a conflict
- Backtrack to the last branch from which the search space has not been explored for both values of a variable ([chronological backtracking](#))
- If no such branch exists, the given CNF formula is unsatisfiable

Conflict Analysis and Backtracking

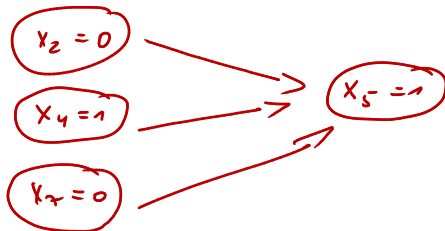


$$F = \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots$$

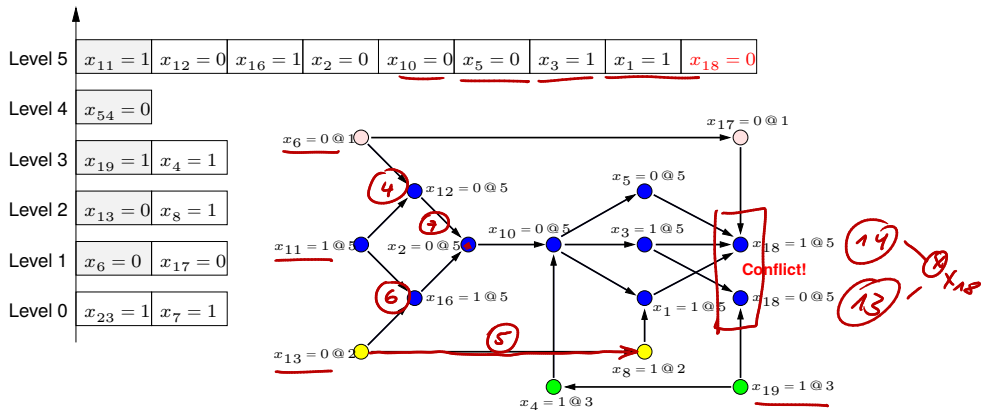
Conflict Analysis and Backtracking

$(x_2, \neg x_3, x_2, \cancel{x_5})$
unit literal

- Data structure for performing the conflict analysis in today's SAT algorithms: **implication graph**
 - Directed, acyclic graph
 - Nodes represent assignments to variables
 - Edges reflect which set of assignments have caused an implication
- The implication graph is updated at every variable assignment and every backtrack operation of the search process



Conflict Analysis and Backtracking: Implication Graph

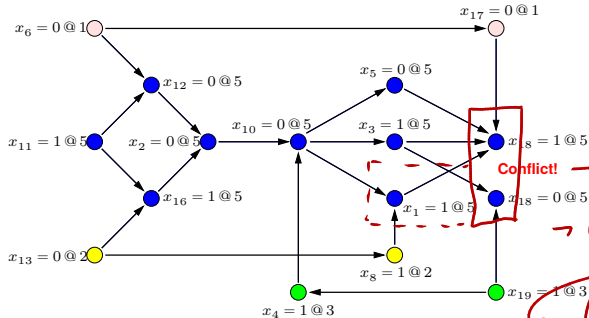


$$F = \underbrace{(x_{23})}_1 \wedge \underbrace{(x_7, \neg x_{23})}_2 \wedge \underbrace{(x_6, \neg x_{17})}_3 \wedge \underbrace{(x_6, \neg x_{11}, \neg x_{12})}_4 \wedge \underbrace{(x_{13}, x_8)}_5 \wedge \underbrace{(\neg x_{11}, x_{13}, x_{16})}_6 \wedge \underbrace{(x_{12}, \neg x_{16}, \neg x_2)}_7 \wedge \underbrace{(x_2, \neg x_4, \neg x_{10})}_8 \wedge \underbrace{(\neg x_{19}, x_4)}_9 \wedge \underbrace{(x_{10}, \neg x_5)}_{10} \wedge \underbrace{(x_{10}, x_3)}_{11} \wedge \underbrace{(x_{10}, \neg x_8, x_1)}_{12} \wedge \underbrace{(\neg x_{19}, \neg x_{18}, \neg x_3)}_{13} \wedge \underbrace{(x_{17}, \neg x_1, x_{18}, \neg x_3, x_5)}_{14} \wedge \dots$$

Conflict Analysis and Backtracking

- During the conflict analysis the implication graph is traversed backwards (in reverse order of the assignments in the decision stack) starting from the conflicting point, to allow to compute the series of resolution steps which finally lead to the **conflict clause**.
- Different termination criteria for interrupting the resolution steps lead to different conflict clauses.
- Implementations:
 - 1UIP (Standard technique explained in the following)
 - RelSat
 - Grasp
 - ...

Conflict Analysis and Backtracking



$$\begin{aligned} & x_{17} = 0 \\ & \wedge x_5 = 0 \\ & \wedge x_3 = 1 \\ & \wedge x_1 = 1 \\ & \wedge x_{19} = 1 \end{aligned}$$

$\Rightarrow \text{conflict!}$

$$\neg (x_{12} = 0 \wedge x_5 = 0 \wedge x_3 = 1 \wedge x_1 = 1 \wedge x_{19} = 1)$$

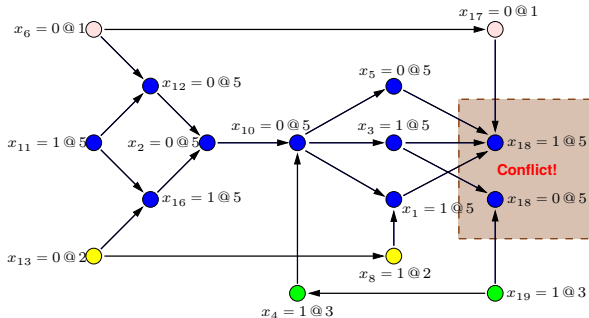
$$= (\neg x_{12} \vee \neg x_5 \vee \neg x_3 \vee \neg x_1 \vee \neg x_{19})$$

$$F = (x_{23}) \wedge (x_7, \neg x_{23}) \wedge (x_6, \neg x_{17}) \wedge (x_6, \neg x_{11}, \neg x_{12}) \wedge (x_{13}, x_8) \wedge (\neg x_{11}, x_{13}, x_{16}) \wedge (x_{12}, \neg x_{16}, \neg x_2) \wedge (x_2, \neg x_4, \neg x_{10}) \wedge (\neg x_{19}, x_4) \wedge (x_{10}, \neg x_5) \wedge (x_{10}, x_3) \wedge (x_{10}, \neg x_8, x_1) \wedge (\neg x_{19}, \neg x_{18}, \neg x_3) \wedge (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \wedge \dots$$

$$(\neg x_{19}, \neg x_3, x_{17}, \neg x_1, x_5)$$

conflict clause

Conflict Analysis and Backtracking

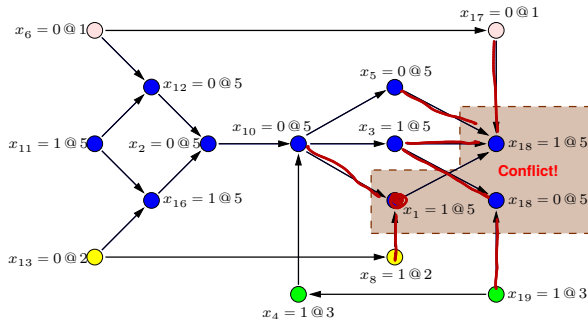


$$F = (x_{23}) \wedge (x_7, \neg x_{23}) \wedge (x_6, \neg x_{17}) \wedge (x_6, \neg x_{11}, \neg x_{12}) \wedge (x_{13}, x_8) \wedge (\neg x_{11}, x_{13}, x_{16}) \wedge (x_{12}, \neg x_{16}, \neg x_2) \wedge (x_2, \neg x_4, \neg x_{10}) \wedge$$

$$(\neg x_{19}, x_4) \wedge (x_{10}, \neg x_5) \wedge (x_{10}, x_3) \wedge (x_{10}, \neg x_8, x_1) \wedge (\neg x_{19}, \neg x_{18}, \neg x_3) \wedge (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \wedge \dots$$

$$R_1 = (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \otimes_{x_{18}} (\neg x_{19}, \neg x_{18}, \neg x_3) = (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19})$$

Conflict Analysis and Backtracking



$$F = (x_{23}) \wedge (x_7, \neg x_{23}) \wedge (x_6, \neg x_{17}) \wedge (x_6, \neg x_{11}, \neg x_{12}) \wedge (x_{13}, x_8) \wedge (\neg x_{11}, x_{13}, x_{16}) \wedge (x_{12}, \neg x_{16}, \neg x_2) \wedge (x_2, \neg x_4, \neg x_{10}) \wedge$$

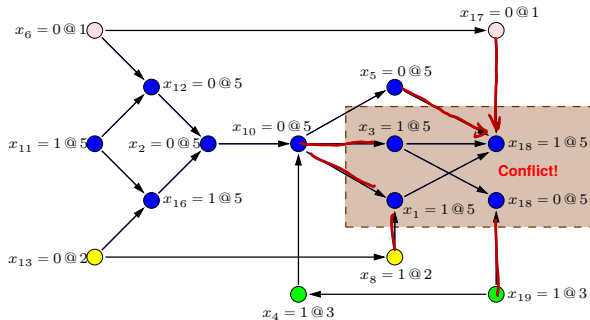
$$(\neg x_{19}, x_4) \wedge (x_{10}, \neg x_5) \wedge (x_{10}, x_3) \wedge (x_{10}, \neg x_8, \boxed{x_1}) \wedge (\neg x_{19}, \neg x_{18}, \neg x_3) \wedge (x_{17}, \neg x_1, \neg x_{18}, \neg x_3, x_5) \wedge \dots$$

$$R_1 = (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \otimes_{x_{18}} (\neg x_{19}, \neg x_{18}, \neg x_3) = (x_{17}, \boxed{\neg x_1}, \neg x_3, x_5, \neg x_{19})$$

$$R_2 = (x_{17}, \boxed{\neg x_1}, \neg x_3, x_5, \neg x_{19}) \otimes_{x_1} (\boxed{x_1}, x_{10}, \neg x_8) = (x_{17}, \neg x_3, x_5, \neg x_{19}, x_{10}, \neg x_8)$$

γ_1

Conflict Analysis and Backtracking



$$F = (x_{23}) \wedge (x_7, \neg x_{23}) \wedge (x_6, \neg x_{17}) \wedge (x_6, \neg x_{11}, \neg x_{12}) \wedge (x_{13}, x_8) \wedge (\neg x_{11}, x_{13}, x_{16}) \wedge (x_{12}, \neg x_{16}, \neg x_2) \wedge (x_2, \neg x_4, \neg x_{10}) \wedge$$

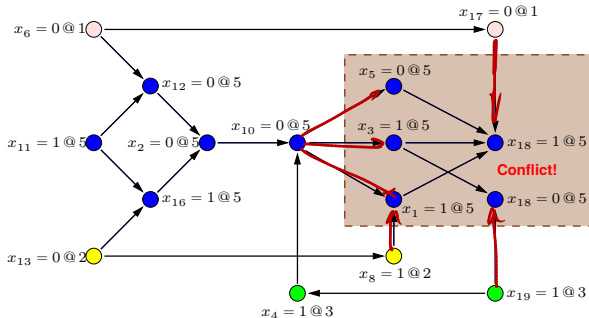
$$(\neg x_{19}, x_4) \wedge (x_{10}, \neg x_5) \wedge (x_{10}, \boxed{x_3}) \wedge (x_{10}, \neg x_8, x_1) \wedge (\neg x_{19}, \neg x_{18}, \neg x_3) \wedge (\boxed{x_{17}}, \neg x_1, x_{18}, \neg x_3, x_5) \wedge \dots$$

$$R_1 = (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \otimes_{x_{18}} (\neg x_{19}, \neg x_{18}, \neg x_3) = (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19})$$

$$R_2 = (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19}) \otimes_{x_1} (x_1, x_{10}, \neg x_8) = (x_{17}, \neg x_3, x_5, \neg x_{19}, x_{10}, \neg x_8)$$

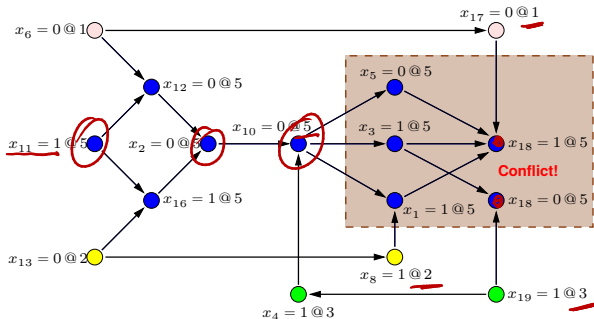
$$R_3 = (\boxed{x_{17}}, \boxed{\neg x_3}, x_5, \neg x_{19}, x_{10}, \neg x_8) \otimes_{x_3} (\boxed{x_{10}}, \boxed{x_3}) = \boxed{x_{17}}, \boxed{x_5}, \neg x_{19}, \boxed{x_{10}}, \neg x_8$$

Conflict Analysis and Backtracking



$$\begin{aligned}
 F &= (x_{23}) \wedge (x_7, \neg x_{22}) \wedge (x_6, \neg x_{17}) \wedge (x_6, \neg x_{11}, \neg x_{12}) \wedge (x_{13}, x_8) \wedge (\neg x_{11}, x_{13}, x_{16}) \wedge (x_{12}, \neg x_{16}, \neg x_2) \wedge (x_2, \neg x_4, \neg x_{10}) \wedge \\
 &\quad (\neg x_{19}, x_4) \wedge (x_{10}, \neg x_5) \wedge (x_{10}, x_3) \wedge (x_{10}, \neg x_8, x_1) \wedge (\neg x_{19}, \neg x_{18}, \neg x_3) \wedge (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \wedge \dots \\
 R_1 &= (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \otimes_{x_{18}} (\neg x_{19}, \neg x_{18}, \neg x_3) = (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19}) \\
 R_2 &= (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19}) \otimes_{x_1} (x_1, x_{10}, \neg x_8) = (x_{17}, \neg x_3, x_5, \neg x_{19}, x_{10}, \neg x_8) \\
 R_3 &= (x_{17}, \neg x_3, x_5, \neg x_{19}, x_{10}, \neg x_8) \otimes_{x_3} (x_{10}, x_3) = (x_{17}, x_5, \neg x_{19}, x_{10}, \neg x_8) \\
 R_4 &= (x_{17}, x_5, \neg x_{19}, x_{10}, \neg x_8) \otimes_{x_5} (x_{10}, \neg x_5) = (x_{17}, \neg x_{19}, x_{10}, \neg x_8)
 \end{aligned}$$

Conflict Analysis and Backtracking



$$F = (x_{23}) \wedge (x_7, \neg x_{23}) \wedge (x_6, \neg x_{17}) \wedge (x_6, \neg x_{11}, \neg x_{12}) \wedge (x_{13}, x_8) \wedge (\neg x_{11}, x_{13}, x_{16}) \wedge (x_{12}, \neg x_{16}, \neg x_2) \wedge (x_2, \neg x_4, \neg x_{10}) \wedge$$

$$(\neg x_{19}, x_4) \wedge (x_{10}, \neg x_5) \wedge (x_{10}, x_3) \wedge (x_{10}, \neg x_8, x_1) \wedge (\neg x_{19}, \neg x_{18}, \neg x_3) \wedge (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \wedge \dots$$

$$R_1 = (x_{17}, \neg x_1, x_{18}, \neg x_3, x_5) \otimes_{x_{18}} (\neg x_{19}, \neg x_{18}, \neg x_3) = (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19})$$

$$R_2 = (x_{17}, \neg x_1, \neg x_3, x_5, \neg x_{19}) \otimes_{x_1} (x_1, x_{10}, \neg x_8) = (x_{17}, \neg x_3, x_5, \neg x_{19}, x_{10}, \neg x_8)$$

$$R_3 = (x_{17}, \neg x_3, x_5, \neg x_{19}, x_{10}, \neg x_8) \otimes_{x_3} (x_{10}, x_3) = (x_{17}, x_5, \neg x_{19}, x_{10}, \neg x_8)$$

$$R_4 = (x_{17}, x_5, \neg x_{19}, x_{10}, \neg x_8) \otimes_{x_5} (x_{10}, \neg x_5) = (x_{17}, \neg x_{19}, x_{10}, \neg x_8) \leftarrow \text{Final conflict clause}$$

Conflict Analysis and Backtracking

Observation: Many partitions of the implication graph into 'conflict side' and 'reason side' are possible.

Question: Which partition should we choose? $\Rightarrow$ 1UIP scheme ...

Definition (Unique Implication Point)

A node in the implication graph is called a *Unique Implication Point (UIP)* iff a paths from the decision variables of the current decision level to the conflict variable go through this node.

Remark: The set of UIPs is not empty. The node with the *decision variable* of the current decision level is a Unique Implication Point.

Conflict Analysis and Backtracking

Definition (1st Unique Implication Point)

The *1st Unique Implication Point (1UIP)* is the UIP that is closest to the conflict variable.

Definition (Asserting Clause)

An *asserting clause* is a clause with exactly one literal from the current decision level.

Observation: A partition with a UIP on the reason side and all successors of the UIP on the conflict side leads to an asserting clause as a conflict clause.

1UIP scheme: Use the 1UIP for constructing an (asserting) conflict clause.

Conflict Analysis and Backtracking

Observations:

- Conflict analysis according to the 1UIP scheme terminates as soon as the currently computed resolvent contains exactly one literal which was assigned at the decision level being analyzed at that moment (the highest), whereas all other literals were assigned at lower decision levels
- Conflict clauses represent combinations of variables that will inevitably lead to a conflict.

Conflict Analysis and Backtracking

Observations:

- Resolution Lemma allows to insert the conflict clauses into the CNF formula, and consequently to “prune” the whole search tree (they prevent the algorithm from running into identical conflicts)
- Compared to others, the 1UIP scheme turned out to be the most powerful (shorter conflicts clauses, more effective pruning, faster runtime)

Conflict Analysis and Backtracking

topmost untouched level

Non-Chronological Backtracking:

- In today's SAT algorithms the backtrack level is only determined by the resulting conflict clause
- The backtrack level matches the maximum decision level among all the literals in the conflict clause except the UIP literal, that means with the exception of the literal assigned at the current decision level, which is conflicting by definition
- Idea: "What would have happened if the conflict clause had already been contained in the original CNF formula?"

second highest level in the conflict clause

Conflict analysis and backtracking

Non-Chronological Backtracking:

- Procedure
 1. Backtrack down to the given backtrack level
 2. Assign the truth value implied by the conflict clause constructed just before (after backtracking, this conflict clause will be automatically a unit clause with the UIP literal as the only unassigned literal)
 3. Proceed with the search process
- If a conflict appears at decision level 0, the CNF formula is unsatisfiable

What Else?

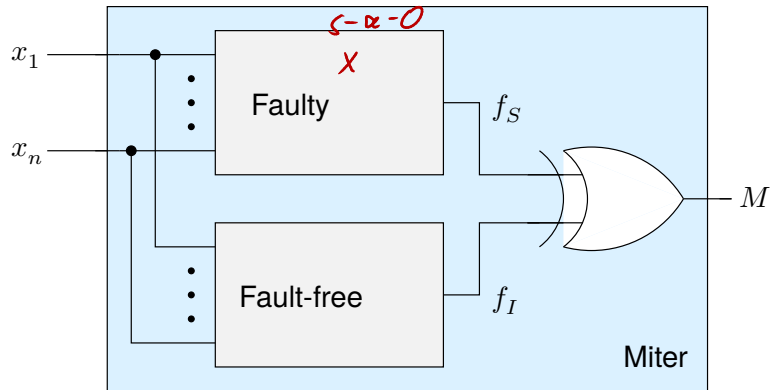
We skipped many (important) topics:

- Deletion of conflict clauses
- Restarts
- Incremental solving
- Certifying UNSAT
- Extensions of SAT: (D)QBF, SMT, MAXSAT, ...

SAT for ATPG



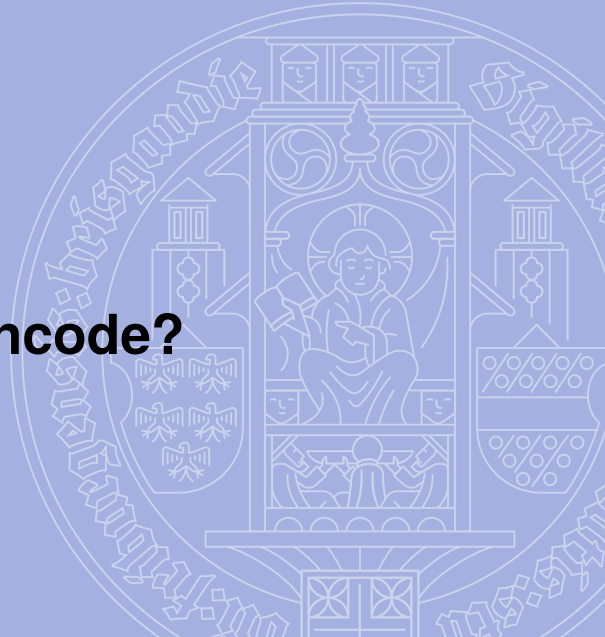
Reminder: Miter Circuit



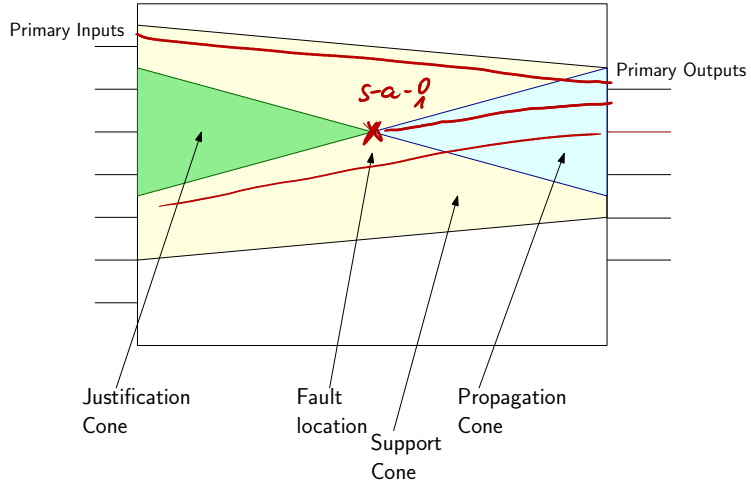
$\Rightarrow M = 1$ for some input assignment $\Leftrightarrow$ Fault-free circuit & faulty circuit are **not** equivalent

SAT for ATPG

What do we have to encode?



Cone-of-Influence



Cone-of-Influence

- The white parts (outside the Support Cone) are irrelevant.
→ Do not encode them at all.
- Support Cone and Justification Cone are identical in the fault-free and the faulty circuit.
→ Merge these parts.
- The Propagation Cone potentially has different values in the two circuits.
→ Need to be encoded twice (fault-free and faulty).

SAT for ATPG

Incremental Solving



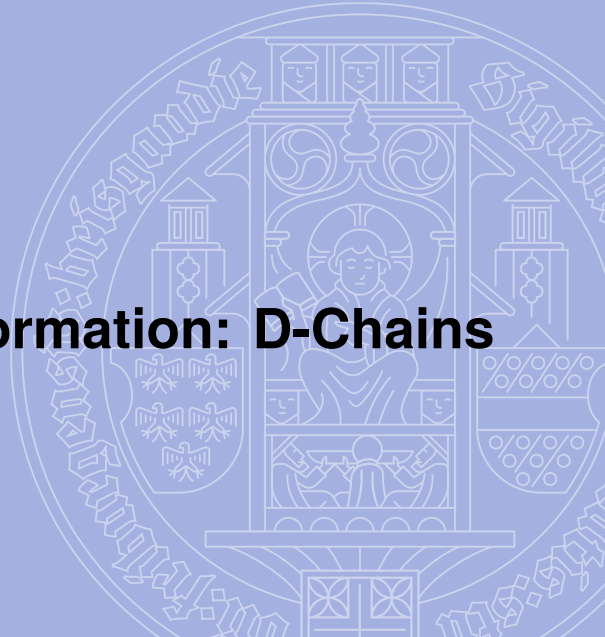
Create Miter Step-by-Step

$O_f = \{o_1, \dots, o_n\}$ relevant circuit outputs.

```
1  $o_1 = O_f.\text{pop}()$ ;
2  $\Phi = \text{encodeMiter}(o_1)$  ;
3  $\text{solveResult} = \text{solve}(\Phi + \text{assumeDifferenceAt}(o_1))$ ;
4 if  $\text{solveResult.isSAT}()$  then
5   |  $\text{return extractTestPattern}(\text{solveResult})$ ;
6 end
7  $o_{last} = o_1$ ;
8 while not  $O_f.\text{empty}()$  do
9   |  $o_i = O_f.\text{pop}()$ ;
10  |  $\Phi += \text{encodePropagationCone}(o_i)$ ;
11  |  $\Phi += \text{encodeSupportCone}(o_i)$ ;
12  |  $\Phi += \text{encodeNoDifferenceAt}(o_{last})$ ;
13  |  $\text{solveResult} = \text{solve}(\Phi + \text{assumeDifferenceAt}(o_i))$ ;
14  | if  $\text{solveResult.isSAT}()$  then
15    |  $\text{return extractTestPattern}(\text{solveResult})$ ;
16  | end
17  |  $o_{last} = o_i$ ;
18 end
19 return “untestable”;
```


SAT for ATPG

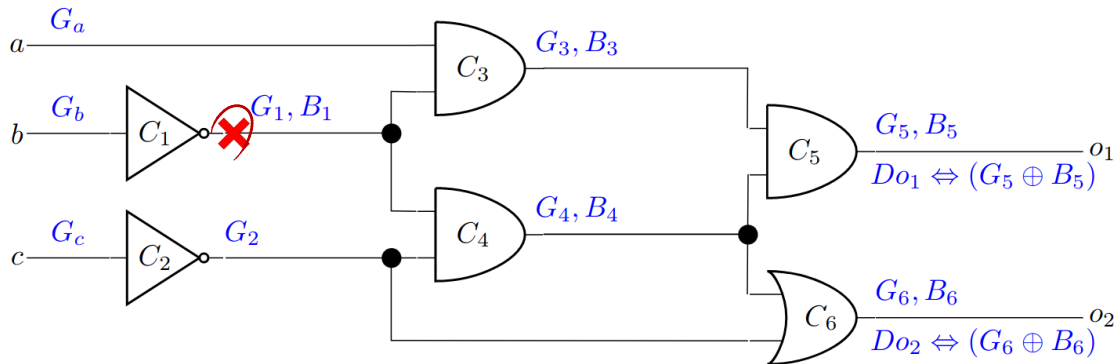
Adding Structural Information: D-Chains



Structural Information

- The structural information between fault-free and faulty circuit is not explicitly available in the formula.
 - Only primary inputs and outputs “connected” in the formula, but not internal signals.
- Add **redundant** clauses (“D-Chains”) for the gates in the **propagation cone** to provide this information.

Available Values in the Miter



G = good (fault-free value), B = bad (faulty) value, D = difference on signals?

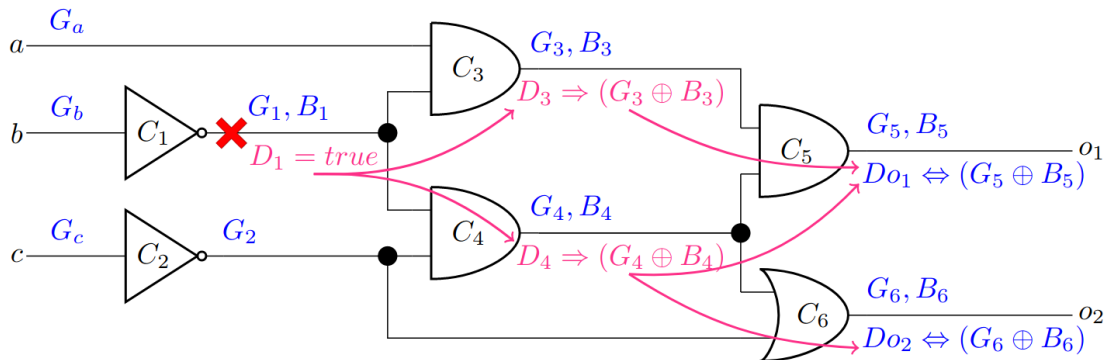
$D o_1 \vee D o_2$

Forward D-Chains

- New Variable D_ℓ for every line in the Propagation Cone
- Enforce $D_\ell = \text{true}$ if ℓ is the fault location.
- Enforce propagation to the output by adding implications (next slide).

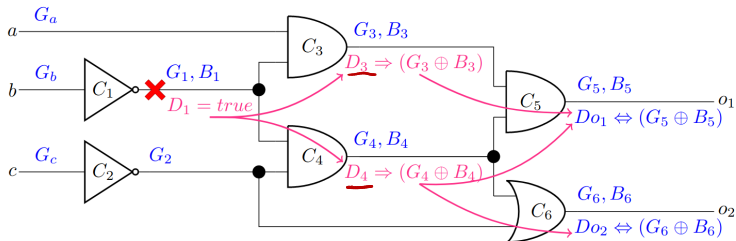
Forward D-Chains

Add the following implications:



Forward D-Chains

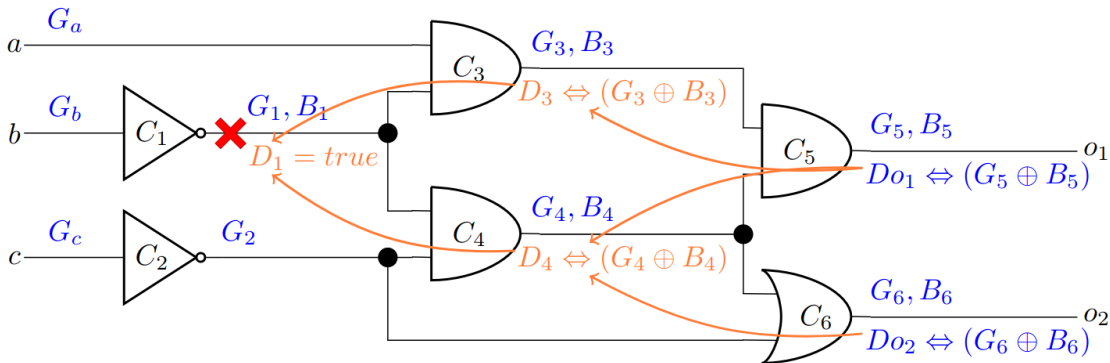
Add the following implications:



- $D_1 = \text{true}$
- $D_3 \Rightarrow (G_3 \oplus B_3)$
- $D_4 \Rightarrow (G_4 \oplus B_4)$
- $D_1 \Rightarrow (D_3 \vee D_4)$
- $D_3 \Rightarrow D_{o_1}$
- $D_4 \Rightarrow (D_{o_1} \vee D_{o_2})$

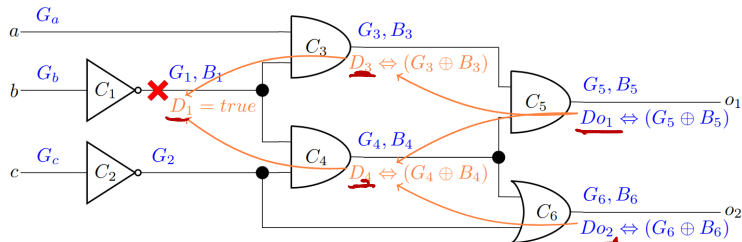
Backward D-Chains

Where there is a different on a gate output, there must be a difference on at least one input as well.



Backward D-Chains

Add the following implications:



- $D_1 = \text{true}$
- $D_3 \Leftrightarrow (G_3 \oplus B_3)$
- $D_4 \Leftrightarrow (G_4 \oplus B_4)$
- $D_{o_1} \Rightarrow (D_3 \vee D_4)$
- $D_{o_2} \Rightarrow D_4$
- $D_3 \Rightarrow D_1$
- $D_4 \Rightarrow D_1$

Combined Backward/Forward D-Chains

Both kinds of D-Chains can be combined to give the SAT-solver the maximum amount of information about the propagation of the difference:

- Two new variables per gate D^f and D^b .
- Replace $D^f = (G \oplus B)$ by $D^f \Rightarrow D^b$ (cheaper).
- In total: 7 clauses per gate:
 - 4 clauses for difference condition for the backward D-Chain
 - 1 clause for $D^f \Rightarrow D^b$
 - 1 clause for the forward implication
 - 1 clause for the backward implication.

Good-Diff D-Chain

Each gate output is represented by three variables: G , B , and D .
It holds:

$$D \Leftrightarrow (G \oplus B)$$

$$G \Leftrightarrow (B \oplus D)$$

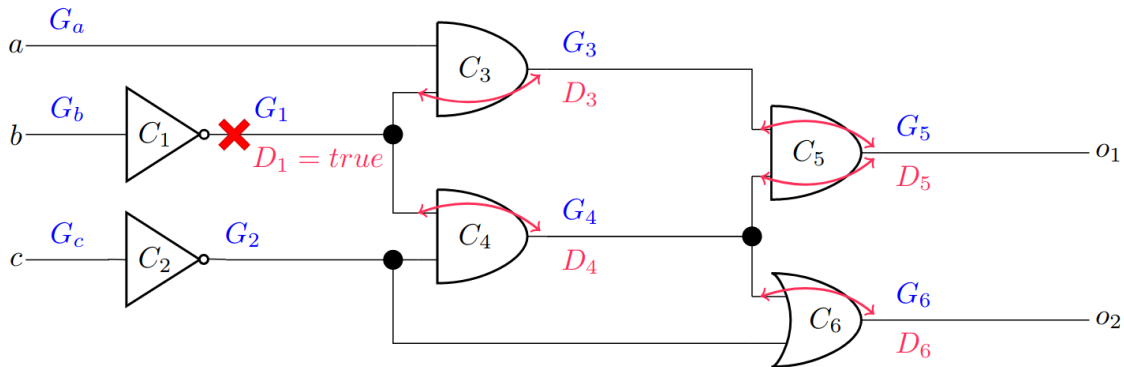
$$B \Leftrightarrow (G \oplus D)$$

This seems redundant, in particular because we only need the D -variables of the circuit outputs.

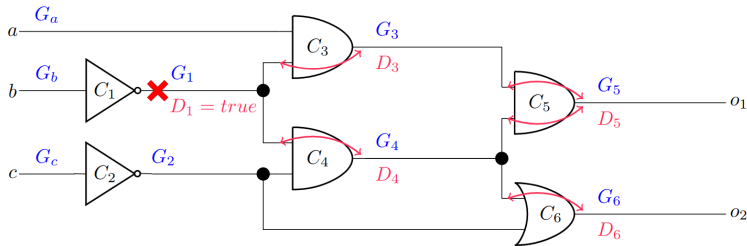
→ Try to get rid of the B variables.

→ Needs a new encoding of the gates for the D variables instead of Tseitin transformation.

Good-Diff D-Chains



Good-Diff D-Chains



- $D_1 = \text{true}$
- $D_3 \Leftrightarrow D_1 \wedge G_a$
- $D_4 \Leftrightarrow D_1 \wedge G_2$
- $D_5 \Leftrightarrow (D_3 \wedge \neg D_4 \wedge G_4) \vee (D_4 \wedge \neg D_3 \wedge G_3) \vee (D_3 \wedge D_4 \wedge (G_3 \equiv G_4))$
- $D_6 \Leftrightarrow D_4 \wedge \neg G_2$

Quite expensive when more than one gate input can carry a difference!

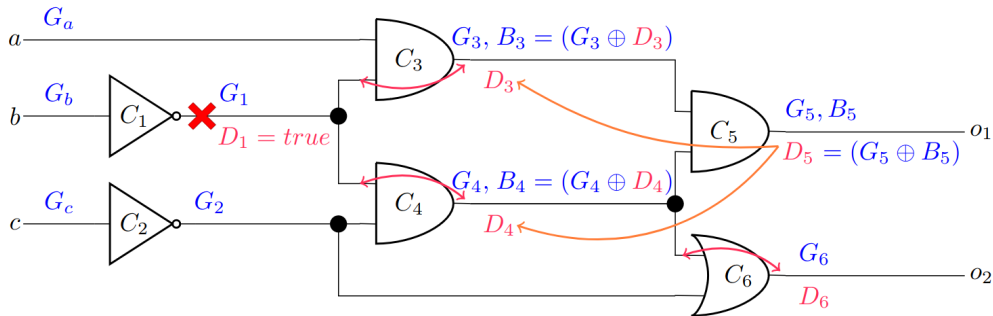
2-input AND, one with diff: 3 clauses, 7 literals

2-input AND, both with diff: 9 clauses, 35 literals

4-input AND, all with diff: 36 clauses

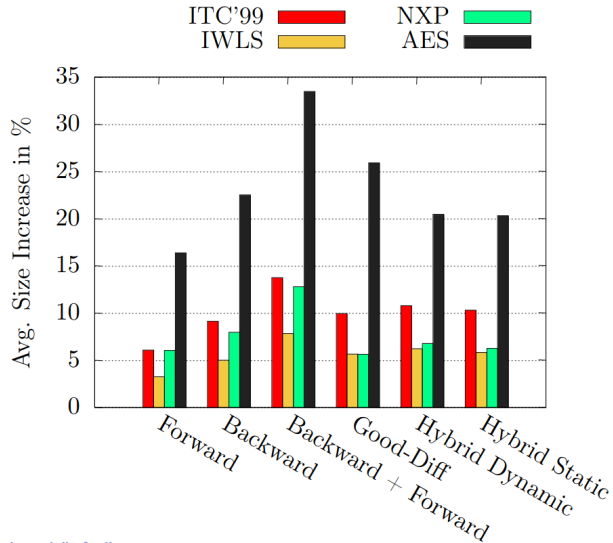
Hybrid D-Chains

- Use Good-Diff D-Chain for gates where its more efficient.
- Use standard Tseitin transformation for all other gates.



- static: Good-Diff only for gates with 1 diff input.
- hybrid: count more precisely.

Results: Change in Formula Size



Results: Change in Solving Time

